Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

ASP.NET Web API Routing

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

ASP.NET Web API Routing

In this article, I am going to discuss **ASP.NET Web API Routing** with examples. Please read our previous article where we discussed **Cross-Origin Resource Sharing in Web API** with examples. The Routing in Web API is one of the most important concepts that you need to understand. Once you understand this concept, then you can easily learn the internal architecture of the ASP.NET Web API pipeline. As part of this article, we are going to discuss the following pointers in detail.

1. **What is Routing in Web API?**
2. **Understanding the Route Table.**
3. **How does the Web API Framework handle an incoming HTTP Request?**

What is Routing in Web API?

The ASP.NET Web API Routing module is responsible for mapping the incoming HTTP requests to a particular controller action method. If you are familiar with the ASP.NET MVC application, then you can easily understand the Routing as it is very much similar to the **ASP.NET MVC Routing.**

The major difference between these two routing mechanisms is that the ASP.NET Web API uses the HTTP method, not the URI path, to select the action. You can also use ASP.NET MVC style routing in Web API which we will discuss in our upcoming articles. ASP.NET Web API Framework supports two types of routing. They are as follows

1. **Convention-based Routing**
2. **Attribute Routing**

Convention-Based Routing in ASP.NET Web API

In Convention-Based Routing, the ASP.NET Web API Framework uses route templates to determine which controller and action method to execute. At least one route template must be added to the route table in order to handle the incoming HTTP requests.

Understanding the Routing Table in ASP.NET Web API:

In the ASP.NET Web API application, a controller is a class that is inherited from ApiController abstract class and this controller class contains action methods that actually handle the incoming HTTP requests. The public methods of the controller class are called action methods or actions. When the ASP.NET Web API Framework receives an HTTP request, it routes that HTTP request to an action method of a controller.

To determine which action method to select or invoke for a particular HTTP request, the ASP.NET Web API Framework uses a Routing table. When we create an ASP.NET Web API application, by default, the Visual Studio creates a default route for our application as shown in the below image. The following route is defined in the WebApiConfig.cs file, which is present inside the App_Start folder.

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Web API configuration and services

        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}
```

Default Route

In the above **WebApiConfig.Register()** method, **config.MapHttpAttributeRoutes()** enables us to use **Attribute Routing in ASP.NET Web API** that we will discuss in our upcoming article. The **config.Routes** is a Route table or route collection of type **HttpRouteCollection**. The "**DefaultApi**" route is added in the route table using **MapHttpRoute()** extension method. The **MapHttpRoute()** extension method internally creates a new instance of **HttpRoute** and adds it to an **HttpRouteCollection**. However, you can create a new route and add it into a collection manually and that we will discuss in our next article.

The Routing table in the ASP.NET Web API application contains each and every route template that we define in the WebApiConfig file. The default route template for the ASP.NET Web API application is "**api/{controller}/{id}**". In this template, the term "**api**" is a literal path segment, and the {controller} and {id} are placeholder variables that will be replaced with the actual value.

When the Route Table is Created in the ASP.NET Web API Application?

The Route table is created only once during the application start. As we know, the Application_Start() event of the Global.asax file is the first method that is going to be executed when our application starts. So, from this method, it will call the Register static method of the WebApiConfig.cs class which will fill the Route table with the defined routes for your application. Now, open the Global.asax file and you will find the following code in it. The point that you need to remember is, the things that you want to configure at the application level at the application startup, you need to configure within the Application_Start() event of the Global.asax file.

```
public class WebApiApplication : System.Web.HttpApplication
{
    protected void Application_Start()
    {
        AreaRegistration.RegisterAllAreas();

        GlobalConfiguration.Configure(WebApiConfig.Register);

        FilterConfig.RegisterGlobalFilters(GlobalFilters.Filters);
        RouteConfig.RegisterRoutes(Route.Table.Routes);
        BundleConfig.RegisterBundles(Bundle.Table.Bundles);
    }
}
```

How does the ASP.NET Web API Framework handle an incoming HTTP Request?

When the ASP.NET Web API Framework receives an HTTP request, it tries to match the URI (Incoming HTTP Request URL) against one of the route templates available in the routing table. If no route template matches the URI, then Web API Framework returns a 404 error to the client who actually makes the request. For example, the following URIs match with the default route template

1. **/api/student**
2. **/api/students/1**
3. **/api/products/pr01**

However, the following URI does not match, because it lacks the "api" segment:
/products/1

The reason for using "**api**" in the route is to avoid collisions between the ASP.NET Web API and MVC routing. So, you can have "**/products**" go to the MVC controller, and "**/api/products**" go to the Web API controller. Of course, if you don't like this convention, you can change the default route table that also we will discuss in our upcoming article.

Once a matching route is found in the Route table, The Web API's Framework then selects the controller and the action. To select the controller, the Web API Framework adds "**Controller**" to the value of the **{controller}** variable that it received in the URI. To find the action, the Web API Framework looks at the HTTP method of the incoming request (i.e. GET, POST, PUT, or DELETE) and then looks for an action method whose name begins with that HTTP method name or method that decorates with the HTTP verb attributes.

For example, with a **GET** request, the Web API Framework looks for an action method that should start with "**Get**", such as "GetProduct" or "GetAllProducts" and the methods that should decorate the GET HTTP verb. This convention only applies to **GET, POST, PUT,** and **DELETE** methods. You can enable other HTTP methods by using attributes on your controller that we will discuss in our upcoming article. Other placeholder variables in the route template, such as **{id},** are mapped to action method parameters.

Example: Routing in ASP.NET Web API Application

Let us see an example for a better understanding of Web API Routing. Suppose you define the following Student controller in your application.

```
public class StudentController : ApiController
{
    public IEnumerable<Student> GetAllStudents()...
    public Student GetStudentById(int id)...
    public HttpResponseMessage DeleteStudent(int id)...
    public HttpResponseMessage PutStudent(Student student)...
}
```

Here are some possible HTTP requests, along with the action that gets invoked for each request.

HTTP Method	URI Path	Action	Parameter
GET	api/student	GetAllStudents	(none)
GET	api/student/2	GetStudentById	4
DELETE	api/student/2	DeleteStudent	4
POST	api/student	(no match)	
PUT	api/student	PutStudent	Student

Notice that the **{id}** segment of the URI, if present, is mapped to the **id** parameter of the action. In our example, the Student controller defines two GET methods, one with an **id** parameter and one with **no parameter**. It also defines one PUT method which takes one parameter of student type from the request body. Here another point you need to understand is that the POST request will fail as the controller does not have any "**Post**" method.

In the next article, I am going to discuss the **Routing variations in ASP.NET Web API Application with examples**. Here, in this article, I try to explain **Routing in ASP.NET Web API** Application with examples. I hope you enjoy this Web API Routing article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

5 thoughts on "ASP.NET Web API Routing"

CHJONES
JUNE 7, 2020 AT 10:40 PM

In the example you have 2 uri calls that look exactly the same. api/student/2
How does it know which one to call? (Get or Delete)?

Reply

UMAIR TAHIR
OCTOBER 14, 2021 AT 8:08 PM

Look at their HTTP Methode. One is delete and other is Get.

Reply

RAKSHA
AUGUST 26, 2020 AT 4:07 PM

This will search for action method name with Get action verb for get call and Delete action verb for Delete call.

Reply

ABHISHEK RANA
JUNE 26, 2021 AT 12:40 PM

Can you please explain this:

You have contradicted 2 statements which confused me:

"The major difference between these two routing mechanisms is that the Web API uses the HTTP method, not the URI path, to select the action" and then you mentioned "When the ASP.NET Web API Framework receives an HTTP request, it tries to match the URI against one of the route templates available in the routing table".

Isn't these 2 statements contradicting each other from URI perspective?

Reply

BENJAMIN
JULY 29, 2021 AT 4:41 AM

"Web API uses the HTTP method" means that it binds HTTP verbs (GET, POST, PUT, DELET) to action methods that contain in their name the HTTP verb (GETallEmployees for example)

While for MVC it is not essential to use HTTP verbs for its route links

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* Email* Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel